# Overview

This project simulates a honeypot environment to observe and analyse unauthorized SSH access attempts. Using the Cowrie honeypot on Ubuntu and an attacking Kali machine, the setup allows safe study of brute-force behaviour, command patterns, and attacker interaction in a controlled environment.

The goal of this project is to:

- Understand common brute-force and unauthorized access techniques
- Capture and analyse attacker behaviour and commands
- Gain hands-on experience with defensive security tools like Cowrie

# Scope

**In Scope**

- Setting up a honeypot environment using Cowrie on Ubuntu.
- Simulating SSH brute-force attacks from a Kali Linux machine.
- Capturing and logging attacker login attempts and commands.
- Basic analysis of attacker behaviour based on collected logs.

**Out of Scope**

- Development of custom scripts or programs for automated attack simulation or log analysis
- Advanced malware analysis or payload execution.
- Integration with SIEM or large-scale monitoring systems.
- Simulation in a real-world environment.
- Defence beyond observation (e.g., automated blocking or response).

# Features

- Simulates an SSH/Telnet service using the Cowrie honeypot.
- Captures SSH brute-force attempts (usernames, passwords).
- Logs all commands executed by attackers during sessions.
- Generates structured log files for analysis of attacker behaviour.

# Setup

- Environment built using two virtual machines inside Oracle VirtualBox:
    - Ubuntu VM (Honeypot): Hosts Cowrie to capture and log attacker activity
    - Kali Linux (Attacker): Acts as an attacker performing SSH login attempts against the honeypot
- Cowrie requires Python 3.10 or higher, so Ubuntu 22.04 was used as it meets the required version by default.
- Kali Linux was cloned from existing setup.
- Cowrie was obtained from official public GitHub repository and installed in Ubuntu VM
- Both virtual machines were configured to use Host-Only Adaptor
- Required dependencies were installed (e.g., git, Python and related libraries)
- SSH brute-force attacks were simulated from the Kali machine using tools such as Hydra
- Cowrie is used for the honeypot as it provides:
    - A controlled, emulated SSH/Telnet environment
    - Detailed logging of all login attempts, commands and interactions
- Attacker behaviour was analysed by reviewing logs generated by Cowrie

# Methodology

**Virtual Machine Setup**

- Ubuntu 22.04 virtual machine was installed and setup for Cowrie honeypot
- Existing Kali setup was cloned to be used as the attacker
- Both virtual machines were configured using a Host-Only Adapter to allow direct communication while maintaining an isolated environment
- Ubuntu VM was temporarily set to NAT Network to download dependencies

**Installation of Cowrie Honeypot**

- Required system dependencies were installed:

```
sudo apt install python3-pip python3-dev libssl-dev libffi-dev build-essential
```

- Cowrie repository was cloned from official GitHub source:

```
git clone <https://github.com/cowrie/cowrie.git>
```

**Environment and Cowrie Setup**

- A Python virtual environment was created and activated:

```
python3 -m venv TESTenv

source TESTenv/bin/activate
```

- Python package manager (pip) was updated and required dependencies were installed:

```
pip3 install --upgrade pip

pip3 install -r requirements.txt
```

- After installation, Ubuntu VM was switched back to Host-Only Adaptor to maintain the isolated environment.
- Cowrie configuration file was initialised:

```
cp etc/cowrie.cfg.dist etc/cowrie.cfg
```

**Cowrie Configuration**

- The default listening port was changed to allow connections from Kali VM:
  - Default: listen_endpoints = tcp:6415:interface=127.0.0.1
  - Updated: listen_endpoints = tcp:2222:interface=0.0.0.0

**Cowrie Execution**

- Start Cowrie: cowrie start
- Real-time log monitoring was performed using:

```
tail -f var/log/cowrie/cowrie.log
```

```
(TESTenv) seed@seedVM:~/cowrie$ tail -f var/log/cowrie/cowrie.log
2026-04-19T16:54:33.138917Z [-] Python Version 3.10.12 (main, Mar  3 2026, 11:56:32) [GCC 11.4.0]
2026-04-19T16:54:33.138936Z [-] Twisted Version 25.5.0
2026-04-19T16:54:33.138942Z [-] Cowrie Version 2.9.17
2026-04-19T16:54:33.138947Z [-] Sensor UUID: 81735134-3bca-11f1-adf4-dbd359b26c80
2026-04-19T16:54:33.139533Z [-] Loaded output engine: jsonlog
2026-04-19T16:54:33.140088Z [twisted.scripts._twistd_unix.UnixAppLogger#info] twistd 25.5.0 (/home/seed/cowrie/TESTenv/bin/python 3.10.12) starting up.
2026-04-19T16:54:33.140140Z [twisted.scripts._twistd_unix.UnixAppLogger#info] reactor class: twisted.internet.epollreactor.EPollReactor.
2026-04-19T16:54:33.143619Z [-] CowrieSSHFactory starting on 2222
2026-04-19T16:54:33.143997Z [cowrie.ssh.factory.CowrieSSHFactory#info] Starting factory <cowrie.ssh.factory.CowrieSSHFactory object at 0x793d28da26e0>
2026-04-19T16:54:33.169762Z [-] Ready to accept SSH connections
```

- Network Configuration
  - Ubuntu IP: 192.xxx.xx.xxx
  - Kali IP: 192.xxx.xx.xxx
- Testing of SSH connection from Kali: ssh root@192.xxx.xx.xxx -p xxxx
- Verified that connection attempts logs were working through Cowrie logs.

# Test Cases

1. **Basic Brute Force using Hydra**
    - A password brute-force attack was simulated using a small password list:
        - 123, password, admin, root, test
    - Attack was executed using Hydra:

```
hydra -l root -P passwords.txt ssh://192.xxx.xx.xxx -s xxxx
```

    - This test simulates automated password guessing against the SSH service.
2. **Direct Credential Attack**
    - Manual login attempts were performed using commonly used usernames:
        - root: ssh root@xxx.xxx.xx.xxx -p xxxx
        - admin: ssh admin@xxx.xxx.xx.xxx -p xxxx
        - user: ssh user@xxx.xxx.xx.xxx -p xxxx
    - Each account was tested with the following passwords:
        - root: root, test, password
        - admin: admin, 123, password
        - user: user, 123, password
    - This test simulates attackers attempting commonly known default credentials.
3. **Successful Login and Command Execution**
    - A successful login was performed using weak credentials:
        - ssh root@192.xxx.xx.xxx -p xxxx
        - Password: test
    - After gaining access, the following commands were executed:

```
uname -a
whoami
ls
cat /etc/passwd
```

    - This test demonstrates attacker behaviour after gaining access to a system.

# Observations/Findings

### First Test: Basic Brute Force using Hydra

```
New connection: 192.xxx.xx.xxx:32850

New connection: 192.xxx.xx.xxx:32864

New connection: 192.xxx.xx.xxx:32866

New connection: 192.xxx.xx.xxx:32888

New connection: 192.xxx.xx.xxx:32896

login attempt [b'root'/b'password'] succeeded

login attempt [b'root'/b'admin'] succeeded

login attempt [b'root'/b'test'] succeeded

login attempt [b'root'/b'123'] succeeded

login attempt [b'root'/b'root'] failed
```

- Multiple rapid connection attempts were observed from the attacker IP (192.xxx.xx.xxx), indicating automated brute-force activity.
- Various password combinations were tested against the root account.
- Several login attempts were successfully accepted by the honeypot due to use of weak credentials.

```
┌──([redacted])-[~]
└─$ hydra -l root -P passwords.txt ssh://192.168.[redacted] -s 2222
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in militar
y or secret service organizations, or for illegal purposes (this is non-binding, thes
e *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-19 16:59:14
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommend
ed to reduce the tasks: use -t 4
[DATA] max 5 tasks per 1 server, overall 5 tasks, 5 login tries (l:1/p:5), ~1 try per
 task
[DATA] attacking ssh://192.168.[redacted]:2222/
[2222][ssh] host: 192.168.[redacted]   login: root   password: password
[2222][ssh] host: 192.168.[redacted]   login: root   password: admin
[2222][ssh] host: 192.168.[redacted]   login: root   password: test
[2222][ssh] host: 192.168.[redacted]   login: root   password: 123
1 of 1 target successfully completed, 4 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-19 16:59:15
```

```
[redacted]
2026-04-19T16:59:13.623230Z [cowrie.ssh.factory.CowrieSSHFactory] New connection: 192.[redacted] (192.[redacted]) [session: [redacted]]
[redacted]
2026-04-19T16:59:13.628261Z [cowrie.ssh.factory.CowrieSSHFactory] New connection: 192.[redacted] (192.[redacted]) [session: [redacted]]
[redacted]
2026-04-19T16:59:13.629015Z [cowrie.ssh.factory.CowrieSSHFactory] New connection: 192.[redacted] (192.[redacted]) [session: [redacted]]
[redacted]
2026-04-19T16:59:13.629866Z [cowrie.ssh.factory.CowrieSSHFactory] New connection: 192.[redacted] (192.[redacted]) [session: [redacted]]
[redacted]
2026-04-19T16:59:13.631669Z [cowrie.ssh.factory.CowrieSSHFactory] New connection: 192.[redacted] (192.[redacted]) [session: [redacted]]
```

```
2026-04-19T16:59:13.667732Z [HoneyPotSSHTransport,2,192.[redacted]] login attempt [b'root'/b'password'] succeeded
2026-04-19T16:59:13.668424Z [HoneyPotSSHTransport,3,192.[redacted]] login attempt [b'root'/b'admin'] succeeded
2026-04-19T16:59:13.668889Z [HoneyPotSSHTransport,4,192.[redacted]] login attempt [b'root'/b'test'] succeeded
2026-04-19T16:59:13.672415Z [HoneyPotSSHTransport,1,192.[redacted]] login attempt [b'root'/b'123'] succeeded
2026-04-19T16:59:13.672995Z [HoneyPotSSHTransport,5,192.[redacted]] login attempt [b'root'/b'root'] failed
```

### Second Test: Direct Credential Attack

```
New connection: 192.xxx.xx.xxx:xxxxx (192.xxx.xx.xxx:xxxx) [session: xx7aec83a5ae]
login attempt [b'root'/b'root'] failed
login attempt [b'root'/b'test'] succeeded
New connection: 192.xxx.xx.xxx:xxxxx (192.xxx.xx.xxx:xxxx) [session: c8dee3d11472]
login attempt [b'admin'/b'admin'] failed
login attempt [b'admin'/b'123'] failed
login attempt [b'admin'/b'password'] failed
New connection: 192.xxx.xx.xxx:xxxxx (192.xxx.xx.xxx:xxxx) [session: 22328958558e]
login attempt [b'user'/b'user'] failed
login attempt [b'user'/b'123'] failed
login attempt [b'user'/b'password'] failed
```

- Three distinct connection attempts were made from the attacker IP (192.xxx.xx.xxx) to the honeypot (192.xxx.xx.xxx) through port xxxx.
- Common usernames (root, admin, user) and weak passwords were tested.
- A successful login was achieved for the root account using the password test, while all attempts for admin and user failed.

```
┌──(jin㉿Jin)-[~]
└─$ ssh root@192.[redacted] -p [redacted]
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
root@192.[redacted]'s password:
Permission denied, please try again.
root@192.[redacted]'s password:

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@svr04:~# exit
Connection to 192.[redacted] closed.

┌──(jin㉿Jin)-[~]
└─$ ssh admin@192.[redacted] -p [redacted]
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
admin@192.[redacted]'s password:
1Permission denied, please try again.
admin@192.[redacted]'s password:
Permission denied, please try again.
admin@192.[redacted]'s password:
admin@192.[redacted]: Permission denied (publickey,password).

┌──(jin㉿Jin)-[~]
└─$ ssh user@192.[redacted] -p [redacted]
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
user@192.[redacted]'s password:
Permission denied, please try again.
user@192.[redacted]'s password:
Permission denied, please try again.
user@192.[redacted]'s password:
user@192.[redacted]: Permission denied (publickey,password).
```

```
2026-04-19T17:21:30.329922Z [cowrie.ssh.factory.CowrieSSHFactory] New connection: 192.168.[redacted] (192.168.[redacted]) [session: [redacted]]
[redacted]
2026-04-19T17:21:38.685788Z [HoneyPotSSHTransport,0,192.[redacted]] login attempt [b'root'/b'root'] failed
[redacted]
2026-04-19T17:21:41.968150Z [HoneyPotSSHTransport,0,192.[redacted]] login attempt [b'root'/b'test'] succeeded
[redacted]
```

```
2026-04-19T17:22:39.147466Z [cowrie.ssh.factory.CowrieSSHFactory] New connection: 192.168.[redacted] (192.168.[redacted]) [session: [redacted]]
[redacted]
2026-04-19T17:22:41.438801Z [HoneyPotSSHTransport,1,192.[redacted]] login attempt [b'admin'/b'admin'] failed
[redacted]
2026-04-19T17:22:44.432073Z [HoneyPotSSHTransport,1,192.[redacted]] login attempt [b'admin'/b'123'] failed
[redacted]
2026-04-19T17:22:47.097163Z [HoneyPotSSHTransport,1,192.[redacted]] login attempt [b'admin'/b'password'] failed
[redacted]
```

### Third Test: Successful Login and Command Execution

New connection: 192.xxx.xx.xxx:xxxxx (192.xxx.xx.xxx:xxxx) [session: 0bc72c71e1cb]

2026-04-19T17:38:28.860277Z [HoneyPotSSHTransport,0,192.xxx.xx.xxx] CMD: uname -a

2026-04-19T17:38:28.861414Z [HoneyPotSSHTransport,0,192.xxx.xx.xxx] Command found: uname -a

2026-04-19T17:38:31.849205Z [HoneyPotSSHTransport,0,192.xxx.xx.xxx] CMD: whoami

2026-04-19T17:38:31.849671Z [HoneyPotSSHTransport,0,192.xxx.xx.xxx] Command found: whoami

2026-04-19T17:38:33.590691Z [HoneyPotSSHTransport,0,192.xxx.xx.xxx] CMD: ls

2026-04-19T17:38:33.592613Z [HoneyPotSSHTransport,0,192.xxx.xx.xxx] Command found: ls

2026-04-19T17:38:37.810867Z [HoneyPotSSHTransport,0,192.xxx.xx.xxx] CMD: cat /etc/passwd

2026-04-19T17:38:37.815978Z [HoneyPotSSHTransport,0,192.xxx.xx.xxx] Command found: cat /etc/passwd

- A successful connection was established by the attacker IP (192.xxx.xx.xxx).
- The commands demonstrate typical attacker reconnaissance behaviour:
    - uname -a : Retrieves basic system information
    - whoami : Identifies the current user and privilege level
    - ls : List files current directory
    - cat /etc/passwd : Displays user account information

```
┌──(jin㉿Jin)-[~]
└─$ ssh root@192.[redacted] -p [redacted]
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
root@192.[redacted]'s password:

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@[redacted]:~# uname -a
Linux [redacted] 3.2.0-4-amd64 #1 SMP Debian 3.2.68-1+deb7u1 x86_64 GNU/Linux
root@[redacted]:~# whoami
root
root@[redacted]:~# ls
root@[redacted]:~# cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
sshd:x:101:65534::/var/run/sshd:/usr/sbin/nologin
[redacted]
root@[redacted]:~#
```

```
2026-04-19T17:38:17.337211Z [cowrie.ssh.factory.CowrieSSHFactory] New connection: 192.[redacted] (192.[redacted]) [session: [redacted]]
[redacted]
2026-04-19T17:38:18.832994Z [HoneyPotSSHTransport,0,192.[redacted]] login attempt [b'root'/b'test'] succeeded
[redacted]
```

```
2026-04-19T17:38:18.852182Z [twisted.conch.ssh.session#info] Getting shell
2026-04-19T17:38:28.860277Z [HoneyPotSSHTransport,0,192.[redacted]] CMD: uname -a
2026-04-19T17:38:28.861414Z [HoneyPotSSHTransport,0,192.[redacted]] Command found: uname -a
2026-04-19T17:38:31.849205Z [HoneyPotSSHTransport,0,192.[redacted]] CMD: whoami
2026-04-19T17:38:31.849671Z [HoneyPotSSHTransport,0,192.[redacted]] Command found: whoami
2026-04-19T17:38:33.590691Z [HoneyPotSSHTransport,0,192.[redacted]] CMD: ls
2026-04-19T17:38:33.592613Z [HoneyPotSSHTransport,0,192.[redacted]] Command found: ls
2026-04-19T17:38:37.810867Z [HoneyPotSSHTransport,0,192.[redacted]] CMD: cat /etc/passwd
2026-04-19T17:38:37.815978Z [HoneyPotSSHTransport,0,192.[redacted]] Command found: cat /etc/passwd
```

# Limitations and Risks

- Misconfigured VM networking could expose the honeypot outside the environment.
- Lack of monitoring beyond logs might make it difficult to catch issues during long-running tests.
- Simple setup that will not work on advanced attackers, reducing realism and accuracy of the results.
- Bugs or oversights in the python scripts that may affect data quality.

## Challenges

- Initial lack of planning and understanding resulting in constant switching between NAT and Host-Only Adaptor modes to install required dependencies.
- Shared clipboard functionality between the host and Ubuntu VM did not initially work, affecting workflow efficiency.
- Resolved by installing VirtualBox Guest Additions and enabling bidirectional clipboard support.
- Dependency installation issues were encountered due to environment inconsistencies and package version conflicts.
- Resolved by using a clean Python virtual environment and updating pip.
- Initial difficulties in configuring the Cowrie environment, particularly with setting up the correct directory structure and configuration files (e.g., missing cowrie.cfg file).
- Resolved by copying the default configuration template.

## Improvements

- Improve initial planning to reduce the need for switching between different network configurations (e.g., NAT and Host-Only modes).
- Implement automated log analysis (e.g., real-time log visualisation) to filter relevant information and identify patterns in attack behaviour for a more efficient analysis.
- Improve the honeypot to include additional services such as HTTP or FTP to capture a wider range of attack behaviours.
- Configure a custom user credentials database to simulate more realistic login scenarios and improve the accuracy of captured interactions.